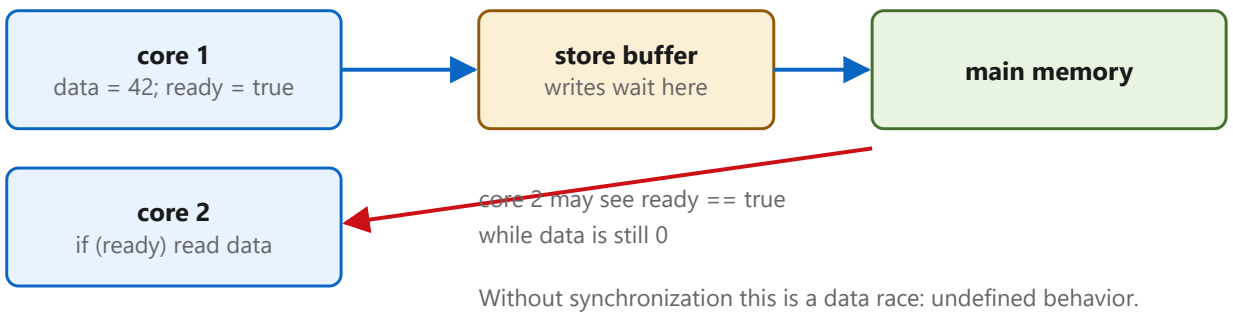
The Memory Model: Orders and Happens-Before

Why one thread can see another's writes late or out of order, release and acquire, the five memory orders, happens-before, the ordering patterns that matter, and thread-safe data structures. Assumes Atomics and Compare-and-Swap.

Why the order matters

The operations in your source are not executed, or observed, in source order across threads. The compiler reorders instructions, and each core has a **store buffer** and caches, so a write by one thread reaches another thread *later*, or even *after a write that came after it*.

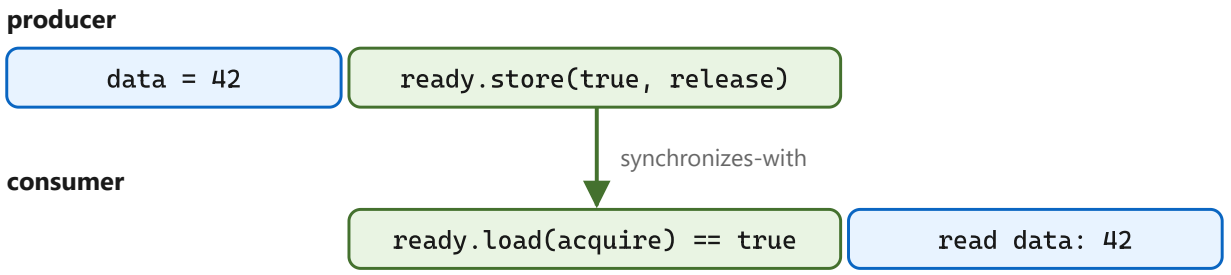


The hardware and the compiler reorder: a plain int gives no promise.

WHO REORDERS	WHY
the compiler	to keep values in registers and to schedule instructions
the CPU	out-of-order execution and store buffers
the cache system	each core sees memory updates at a different time

Atomics let you say **how much ordering you need** between this operation and the others.

Release and acquire: the one rule to learn



everything the producer wrote before the release happens-before everything the consumer does after the acquire.

Release and acquire create a happens-before edge between two threads.

```
int data = 0;
std::atomic<bool> ready{false};

// producer thread
data = 42;
ready.store(true, std::memory_order_release);

// consumer thread
while (!ready.load(std::memory_order_acquire)) { }
std::cout << data;                          // guaranteed to print 42
```

- A **release store** publishes: every write the thread made *before* it becomes visible to any thread that reads the stored value with an **acquire load**.
- The release/acquire pair **synchronizes-with**, and that creates a **happens-before** edge: `data = 42` happens before the consumer's read, so the read is not a data race and sees 42.
- **happens-before** is the formal "must be visible" relation. Within one thread it is program order; across threads it comes only from synchronization (atomics, mutexes, `join`, futures).

What the consumer can print in the publish-with-a-flag example above, for each pair of memory orders:

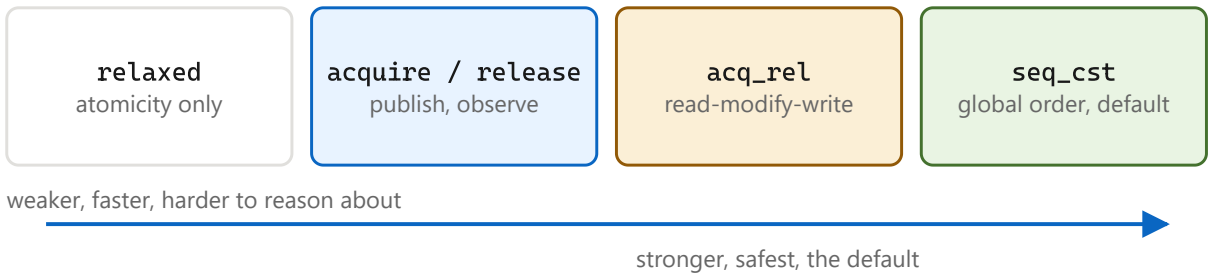
`READY.STORE(TRUE, ...)`	`READY.LOAD(...)`	WHAT THE CONSUMER PRINTS
`release` (or `seq_cst`)	`acquire` (or `seq_cst`)	always **42**
`relaxed`	`acquire`	may print **0**: the flag can become visible before the data
`release`	`relaxed`	may print **0**: nothing guarantees the data is visible after the load
`relaxed`	`relaxed`	may print **0**

- Both sides must take part: a release store read by a **relaxed** load publishes nothing.
- A **fence** (`std::atomic_thread_fence`) gives the ordering without tying it to one variable. Know it exists; reach for acquire/release on the atomic first.
- Without any synchronization, two threads touching the same non-atomic variable (one writing) is a **data race: undefined behaviour**, not merely "stale data".

> KEY POINT "What does acquire/release guarantee?" Everything written before the release is visible after the matching acquire. Say *synchronizes-with* and *happens-before*, and that `seq_cst` is the safe default.

The five memory orders

There are five orders. Atomic operations take one as an optional argument; the default is the strongest.



Start with the strongest, seq_cst, and weaken only with a reason.

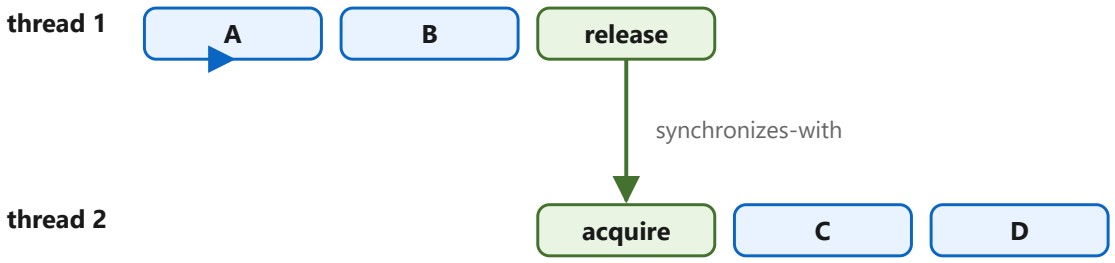
ORDER	GUARANTEE	USE
`relaxed`	only atomicity (and a single total order for *that* variable)	counters and statistics nobody synchronizes on
`acquire` (loads)	nothing after it moves before it; sees what the matching release published	reading a flag or pointer that publishes data
`release` (stores)	nothing before it moves after it; publishes earlier writes	setting that flag or pointer
`acq_rel` (read-modify-write)	both at once	`fetch_sub` on a reference count, a lock hand-over
`seq_cst` (default)	acquire/release **plus** one global order of all `seq_cst` operations that every thread agrees on	when in doubt: the only order that is easy to reason about

X86 HIDES BUGS x86 already gives release/acquire for ordinary loads and stores, so wrong orderings often work on a developer's PC and fail on ARM (phones, Apple silicon, many servers). The simulator above shows what the **C++ model allows**, not what x86 does.

- Both sides must take part: a release store read by a **relaxed** load publishes nothing.
- A **fence** (`std::atomic_thread_fence`) gives the ordering without tying it to one variable. Know it exists; reach for acquire/release on the atomic first.
- Without any synchronization, two threads touching the same non-atomic variable (one writing) is a **data race: undefined behaviour**, not merely stale data.

Happens-before

Happens-before is the formal rule that decides what a thread must see. Inside one thread it is just program order. Between threads it exists *only* because of synchronization: a release/acquire pair, a mutex unlock followed by a lock, thread start and join, a future becoming ready.



A, B happen-before C, D. Within one thread: program order. Between threads: only through synchronization.

If A happens-before B, A's effects are visible to B.

SYNCHRONIZATION	CREATES HAPPENS-BEFORE BETWEEN
release store, acquire load of the same atomic	everything before the store and everything after the load
`mutex.unlock()`, then `lock()` by another thread	the critical sections
`std::thread` constructor	the code before it and the start of the thread function
`join()`	the end of the thread and the code after the join
`promise.set_value()`, `future.get()`	the value and its reader

KEY POINT "What does acquire/release guarantee?" Everything written before the release is visible after the matching acquire. Say *synchronizes-with* and *happens-before*, and that `seq_cst` is the safe default.

Ordering patterns that matter



Why: adding a reference orders nothing. Dropping the last must see all the other threads' writes before deleting.

Compiled: five owners on four threads; the object was destroyed exactly once.

Each pattern picks the weakest order that still creates the edge it needs.

PATTERN	ORDERS	WHY
Publish with a flag or pointer	store `release`, load `acquire`	the data written before the store is visible after the load
Statistics counter	`relaxed` `fetch_add`	only the final total matters; no other data depends on it
Reference count	increment `relaxed`, decrement `acq_rel`	adding a reference needs no ordering; the thread that drops the last one must see every other thread's writes before it deletes
Spinlock	lock `acquire`, unlock `release`	a critical section must not leak out of the lock in either direction
Shutdown flag	`relaxed` if nothing else depends on it, else `release` / `acquire`	a flag alone only needs to be seen eventually
SPSC queue indices	own index `relaxed`, other's `acquire`, publish `release`	a single-producer, single-consumer ring buffer

```
// like shared_ptr's control block
struct Counted {
    std::atomic<int> refs{1};
    void addRef()  { refs.fetch_add(1, std::memory_order_relaxed); }
    void release() {
        // the last one out sees everyone's writes …
        if (refs.fetch_sub(1, std::memory_order_acq_rel) == 1)
            // … before destroying the object
            delete this;
    }
};
```

Compiled: five owners on four threads, the object was destroyed exactly once; a 2,000-round release/acquire publish test read a wrong value 0 times.

> **BE HONEST ABOUT WHAT YOU KNOW** Start with a mutex or `seq_cst`, measure, and weaken orders only when you can say which happens-before edge each one provides. "I use acquire/release for publication, relaxed for counters" is a sound rule of thumb.

Double-checked locking: the classic trap

Double-checked locking is the classic place where a wrong order breaks a program: the pointer can be seen before the object it points to is fully built.

```
std::atomic<Widget*> inst{nullptr};
std::mutex m;
Widget* get() {
    // 1st check, no lock
    Widget* p = inst.load(std::memory_order_acquire);
    if (!p) {
        std::lock_guard lk(m);
        // 2nd check, under the lock
        p = inst.load(std::memory_order_relaxed);
        // publish a FULLY built object
        if (!p) { p = new Widget; inst.store(p, std::memory_order_release); }
    }
    return p;
}
```

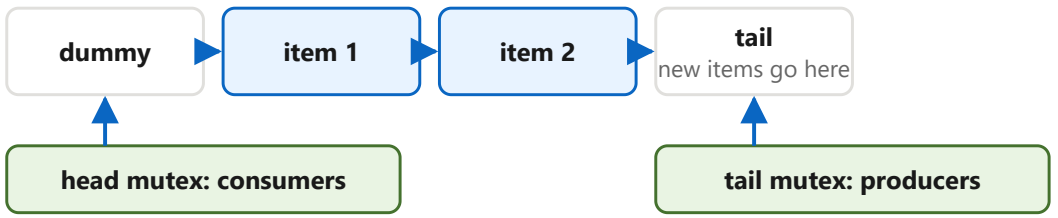
Without the release/acquire pair, another thread could see the pointer before the constructor's writes. In practice use a function-local `static` or `std::call_once` and let the compiler do this.

> **BE HONEST ABOUT WHAT YOU KNOW** Start with a mutex or `seq_cst`, measure, and weaken orders only when you can say which happens-before edge each one provides.

Thread-safe data structures

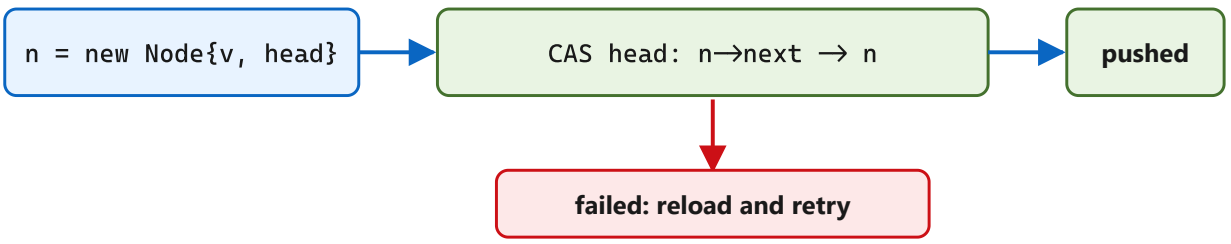
Three levels, in the order you should try them:

LEVEL	HOW	EXAMPLE
1. One coarse lock	a mutex around the whole structure	a stack or map with a mutex (see the interface races in Multithreading)
2. Fine-grained locks	different mutexes for different parts, so threads that touch different parts do not wait	a queue with a head mutex and a tail mutex; a hash table with one mutex per bucket
3. Lock-free	atomics and CAS loops, no mutex	a Treiber stack, an SPSC ring buffer



Compiled: a producer pushing 1 to 1000 and a consumer popping them summed to 500500.

Fine-grained locking: two locks for two independent ends.



Compiled: four threads pushing 1000 nodes each gave 4000 nodes, no lock.

The push is easy; pop needs memory reclamation and ABA protection.

Read-mostly: a lookup table

A price table read by many threads and updated rarely: `std::shared_mutex` with `shared_lock` for readers and `unique_lock` for writers (Other mutex types). Return **copies** or small values, never references into the table.

> **LOCK-FREE IS NOT FASTER BY DEFAULT** A contended CAS loop can be slower than a mutex, and a bug shows up once a month on a customer machine. Use a mutex first, measure, and use a proven library (or the SPSC ring when one producer and one consumer is enough) before writing your own.

- **Wait-free** (every thread finishes in a bounded number of steps) is stronger than **lock-free** (some thread always makes progress) which is stronger than **obstruction-free**.
- Design the **interface** first: whole operations (`tryPop` returning an `optional`), not `empty()` then `pop()`.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Release and acquire	C++ Concurrency in Action	5.1 Memory model basics, and 5.3 Synchronizing operations through the memory ordering overview	148 to 151 and 165 to 170
The five memory orders	C++ Concurrency in Action	5.1 Memory model basics, and 5.3 Synchronizing operations through the memory ordering overview	148 to 151 and 165 to 170
Thread-safe data structures	C++ Concurrency in Action	6.1 to 6.2.2 Designing for concurrency, a thread-safe stack and a queue with condition variables	197 to 206